Part A

Overview

In this part, students are required to design and implement a complete automation testing framework using **Java** and **Selenium WebDriver**, following the same tools and approach demonstrated in the lab sessions. The goal is to apply advanced testing concepts including Page Object Model (POM), Data-Driven Testing and Test Reporting, while maintaining clean, scalable and maintainable code.

Application Under Test

All automation must be executed on the following website:
http://tutorialsninja.com/demo/index.php?route=common/home

Assignment Requirements

- Students must build a structured automation framework using Java, Selenium WebDriver and TestNG, strictly following the same stack used during the lab. The implementation must follow the Page Object Model (POM) design pattern
- All test scenarios provided in the attached Excel sheet must be fully automated. Each row represents a separate test case that must be executed.
- The framework must support running the same test multiple times using different data inputs by reading test data directly from the Excel sheet. Hardcoding test data inside the code is not allowed
- Students are also required to externalize configuration data using a properties file such as URL and browser type. All configuration values must be read dynamically
- All locators must be strong and reliable, using unique attributes such as ID, name, or well-structured CSS selectors. Weak or absolute XPath expressions are not acceptable
- The project must include integration with Allure Reporting to generate a clear and interactive test report, including logs and screenshots on failure. The framework should also be modular, reusable, and follow clean coding standards.

Part B

Overview

In this assignment, you will perform API testing for a live API called **DummyJSON** using **Postman**. You will write automated test scripts, manage authentication tokens, and execute a collection of API tests. Your goal is to build an automated test suite in Postman to ensure the backend is reliable, secure, and performant.

API: https://dummyjson.com

Objectives

- **CRUD Operations**: Validate the lifecycle of a resource (Create, Read, Update, Delete)
- **Automated Assertions**: Use the Postman Test scripts to validate API responses
- **Dynamic Data:** Use Environment Variables to pass data between requests.
- **Authentication:** Simulate a user login and handle.

Assignment Requirements

- Create a workspace and name it A2-TeamLeaderID.
- Inside the workspace, create two folders and create a new request for each of the tests below
- Create environment with the below variables

	Type	**Tasks**	**Validation (tests to be added in script section)**
1	GET	Choose 3 different GET product requests (3 different options)	- Status code 200 - Returned JSON is correct - Response time < limit
2	POST	Add a product using raw JSON data	- Status code 201 - Response body contains the same title you sent - Response body contains an id that is a number
3	POST	Login using password and email added from form-data	- Status code is 200 - Response body contains an access Token (string)

	Type	Tasks	Validation (tests to be added in script section)
4	Env Var	Save the token to an environment variable auth_token	- Create an environment and save the token using script function pm.environment.set()
5	Get	Get current user from the authentication token saved from test 4	- Status code is 200 - Response body contains the correct user information and user id
6	PUT	Update a Product info and change product title and price	- Status code 200 - returned JSON matches your updated value
7	DELETE	Delete a specific product by its ID	- Status code 200 - returned JSON matches your updated value - Returned JSON contain isDeleted set to True (or similar)
8	Negative testing	Get product with invalid data	- Status code 404 Not Found is returned

Environment

Create an **Environment** called DummyJSON - [TeamLeaderID] with the following variables:

	Variable Name	Value	Purpose
1	base_url	DummyJSON URL	- You must use this variable in every request URL
2	auth_token	Leave blank	- **Automatically** filled in user login test

Part A Deliverables

Students are expected to submit

- A complete and working automation project that includes the full source code, execution of all test cases from the Excel sheet
- A generated Allure report demonstrating the test results

Part B Deliverables

- Exported JSON file of the postman collection with your tests
- You will submit a report of the following
    - A screenshot(s) for each required test showing the request details (parameters, body, etc.)
    - A screenshot after running the collection. Must show all requests with green "Pass" badges
- Please note that if the collection doesn't include the automated tests in the "Scripts" section, you will get ZERO for Part B even if you tested manually and added the screenshots

Logistics

- You should work in team of 4 members (preferably the same team as Assignment1)
- Choose your teammates wisely, as we won't accept excuses regarding team members not working together
- Individual Accountability: All members must be able to explain any part of the project upon request
- All Team members should be from the same lab, at least the same TA
- Deliver one zip file that contains two folders one for each part, should be named as such --> stud1ID-stud2ID-Assignment1.zip

Things to consider

- Fail to name the zip file as mentioned above, marks will be deducted
- Cheaters will get negative marks

Policy Regarding Plagiarism:

Students have collective ownership and responsibility of their project. Any violation of academic honesty will have severe consequences and punishment for ALL team members